Ψηφιακά Συστήματα VLSI

3η Εργαστηριακή Άσκηση

Ομάδα 32: Νικόλαος Λοβέρδος (03120442)

Δημήτριος Μιχαλακόπουλος (03120229)

Βάρδια Γ: 11:30-12:30

Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ζητούμενο 1: Synchronous Full Adder

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity sync_full_adder is
5       Port ( A, B, Cin, Clk : in  std_logic;
6              Sum, Cout : out  std_logic);
7   end sync_full_adder;
8
9   architecture behavioral of sync_full_adder is
10      signal wire : std_logic_vector (2 downto 0);
11      begin
12      wire(2) <= A;
13      wire(1) <= B;
14      wire(0) <= Cin;
15
16      process (wire, Clk)
17      begin
18          if Clk'event and Clk='1' then
19              if (wire = "001" or wire = "010" or wire = "100" or wire = "111") then
20              Sum <= '1';
21              else
22              Sum <= '0';
23              end if;
24
25              if (wire = "011" or wire = "101" or wire = "110" or wire = "111") then
26              Cout <= '1';
27              else
28              Cout <= '0';
29              end if;
30          end if ;
31      end process;
32  end behavioral;
```

Ο παραπάνω κώδικας υλοποιεί ένα σύγχρονο πλήρη αθροιστή που λαμβάνει δύο bit εισόδου, το A και το B, μαζί με το κρατούμενο εισόδου Cin και παράγει δύο bit εξόδου, το Sum και το Cout, σύγχρονα με κάθε θετικό παλμό του ρολογιού.

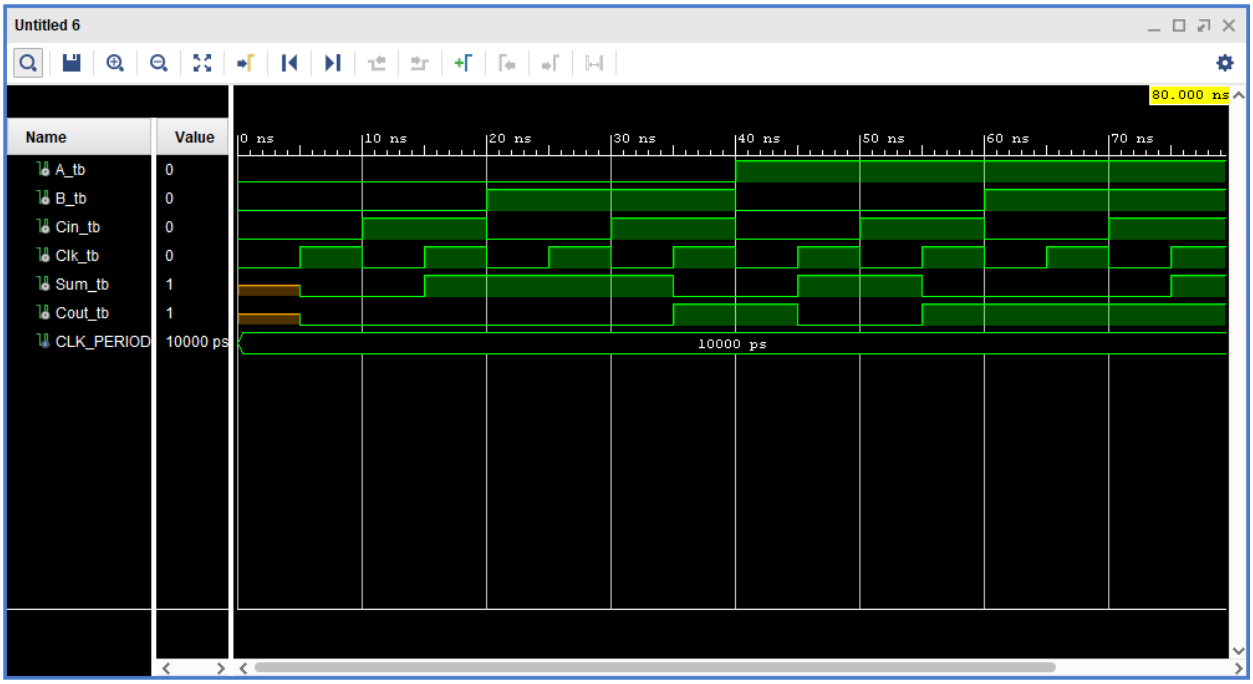
Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity full_adder_tb is
5   end full_adder_tb;
6
7   architecture tb_arch of full_adder_tb is
8
9       component sync_full_adder
10          Port ( A, B, Cin, Clk : in  std_logic;
11                 Sum, Cout : out  std_logic);
12      end component;
13
14      signal A_tb, B_tb, Cin_tb, Clk_tb, Sum_tb, Cout_tb : std_logic;
15      constant CLK_PERIOD : time := 5 ns;
16
17  begin
18
19      DUT: sync_full_adder port map (
20          A => A_tb,
21          B => B_tb,
22          Cin => Cin_tb,
23          Clk => Clk_tb,
24          Sum => Sum_tb,
25          Cout => Cout_tb
26      );
27
28      clk_process: process
29      begin
30          while true loop
31              Clk_tb <= '0';
32              wait for CLK_PERIOD / 2;
33              Clk_tb <= '1';
34              wait for CLK_PERIOD / 2;
35          end loop;
36      end process;
37
```

```
38      stimulus_proc: process
39      begin
40          for i in 0 to 7 loop
41              for i in std_logic range '0' to '1' loop
42                  A_tb <= i;
43                  for j in std_logic range '0' to '1' loop
44                      B_tb <= j;
45                      for k in std_logic range '0' to '1' loop
46                          Cin_tb <= k;
47                          wait for 10 ns;
48                      end loop;
49                  end loop;
50              end loop;
51          end loop;
52          wait;
53      end process;
54  end tb_arch;
```

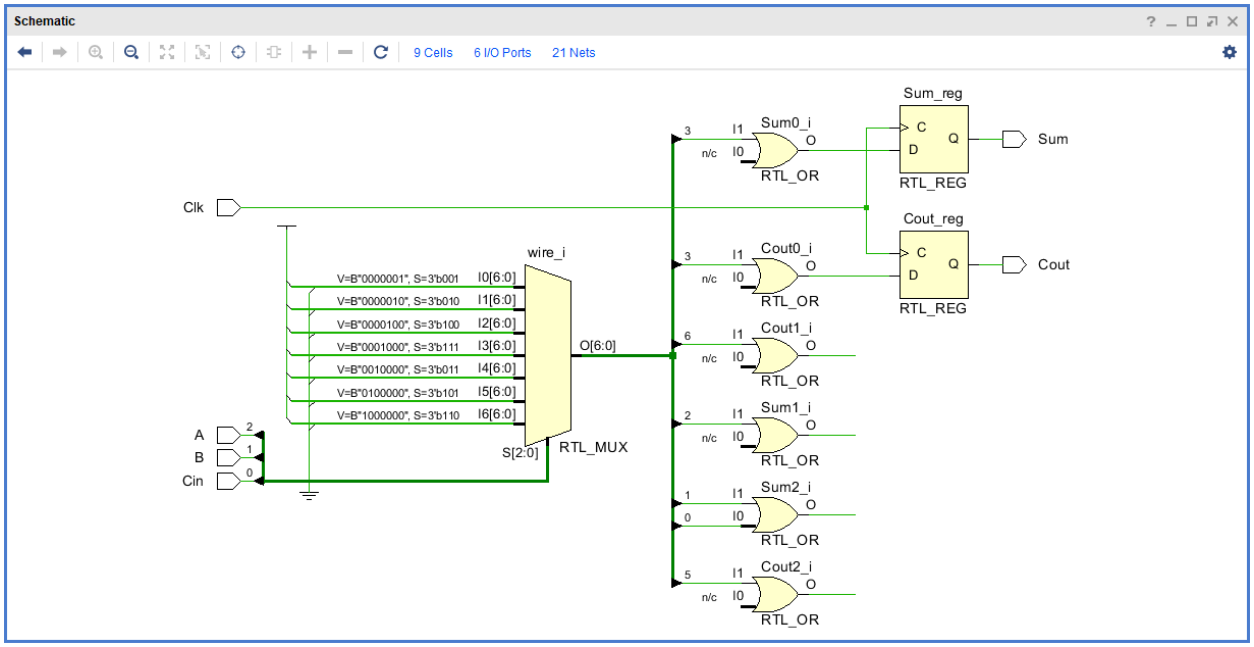
Το παραπάνω testbench εφαρμόζει στον σύγχρονο πλήρη αθροιστή όλους τους διαφορετικούς συνδυασμούς εισόδων μέσω τριών for loops προκειμένου να εξετάσουμε τις αντίστοιχες εξόδους του.

Η προσομοίωση του Synchronous Full Adder φαίνεται παρακάτω:



Παρατηρούμε ότι για κάθε τιμή των εισόδων A, B και Cin του πλήρους αθροιστή, οι έξοδοι Sum και Cout, σε κάθε θετική ακμή του ρολογιού, παίρνουν κάθε φορά την αναμενόμενη τιμή, άρα το κύκλωμα του πλήρους αθροιστή λειτουργεί ορθά.

Το RTL του Synchronous Full Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του Synchronous Full Adder ξεκινάει από την είσοδο Cout_reg/C και καταλήγει στην έξοδο Cout με συνολική καθυστέρηση 4.076 ns. Παρουσιάζει, όμως, ίδια συνολική καθυστέρηση με το μονοπάτι που ξεκινάει από την είσοδο Sum_reg/C και καταλήγει στην έξοδο Sum.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	2	2	1	Cout_reg/C	Cout	4.076	3.276	0.800
Path 2	∞	2	2	1	Sum_reg/C	Sum	4.076	3.276	0.800
Path 3	∞	2	3	2	B	Sum_reg/D	1.932	1.132	0.800
Path 4	∞	2	3	2	A	Cout_reg/D	1.906	1.106	0.800

Ζητούμενο 2: 4-bit Synchronous Full Adder

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity Sync_Full_Adder_4bit is
    Port ( A, B : in  std_logic_vector (3 downto 0);
           Cin : in  std_logic;
           Clock : in std_logic;
           Sum : out  std_logic_vector (3 downto 0);
           Cout : out std_logic);
end Sync_Full_Adder_4bit;

architecture Pipeline of Sync_Full_Adder_4bit is

    component sync_full_adder
        Port ( A, B, Cin, Clk : in  std_logic;
               Sum, Cout : out  std_logic);
    end component;

    component DFF
        Port ( D, Clk : in std_logic;
               Q : out std_logic);
    end component;

    signal a1, b1, s2: std_logic;
    signal a2, b2, s1: std_logic_vector (1 downto 0);
    signal a3, b3, s0, c : std_logic_vector(2 downto 0);

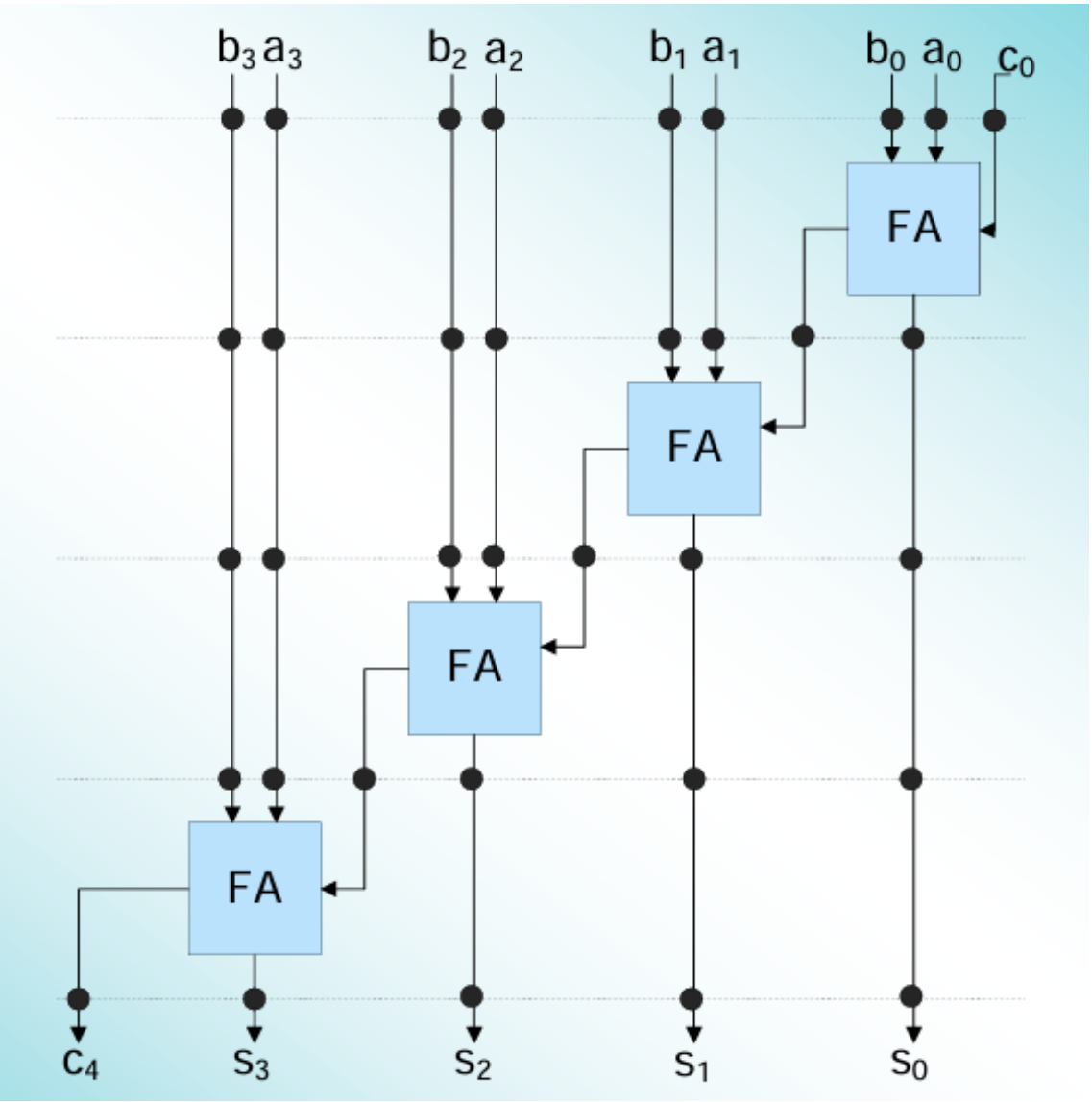
begin
    --STAGE 0
    FA0: sync_full_adder port map (A(0), B(0), Cin, Clock, s0(0), c(0));
    D0a1: DFF port map (A(1), Clock, a1);
    D0b1: DFF port map (B(1), Clock, b1);
    D0a2: DFF port map (A(2), Clock, a2(0));
    D0b2: DFF port map (B(2), Clock, b2(0));
    D0a3: DFF port map (A(3), Clock, a3(0));
    D0b3: DFF port map (B(3), Clock, b3(0));

    --STAGE 1
    D1s0: DFF port map (s0(0), Clock, s0(1));
    FA1: sync_full_adder port map (a1, b1, c(0), Clock, s1(0), c(1));
    D1a2: DFF port map (a2(0), Clock, a2(1));
    D1b2: DFF port map (b2(0), Clock, b2(1));
    D1a3: DFF port map (a3(0), Clock, a3(1));
    D1b3: DFF port map (b3(0), Clock, b3(1));

```

```
46      --STAGE 2
47      D2s0: DFF port map (s0(1), Clock, s0(2));
48      D2s1: DFF port map (s1(0), Clock, s1(1));
49      FA2: sync_full_adder port map (a2(1), b2(1), c(1), Clock, s2, c(2));
50      D2a3: DFF port map (a3(1), Clock, a3(2));
51      D2b3: DFF port map (b3(1), Clock, b3(2));
52
53      --STAGE 3
54      D3s0: DFF port map (s0(2), Clock, Sum(0));
55      D3s1: DFF port map (s1(1), Clock, Sum(1));
56      D3s2: DFF port map (s2, Clock, Sum(2));
57      FA3: sync_full_adder port map (a3(2), b3(2), c(2), Clock, Sum(3), Cout);
58  end Pipeline;
```

Η υλοποίηση αυτού του αθροιστή βρίσκεται στις διαφάνειες του μαθήματος και έγινε με βάση το παραπάνω σχηματικό:



Ο παραπάνω κώδικας, λοιπόν, υλοποιεί έναν σύγχρονο πλήρη αθροιστή 4-bit με χρήση της τεχνικής Pipeline. Χωρίζουμε το κύκλωμα σε τέσσερα επίπεδα και βασιζόμαστε στους χρονισμούς που υπάρχουν στο παραπάνω σχήμα. Έτσι, σε κάθε επίπεδο χρησιμοποιούμε ένα σύγχρονο πλήρη αθροιστή για να χειριστούμε ένα bit της πρόσθεσης και με D flip-flops περνάμε όσες πληροφορίες χρειάζονται στα επόμενα επίπεδα, είτε αυτές είναι είσοδοι είτε είναι έξοδοι των FA. Συνολικά, εν τέλει, χρησιμοποιήσαμε 4 Synchronous Full Adders για τα 4 bit του αθροιστή και 18 D flip-flops για πετύχουμε τις επιθυμητές καθυστερήσεις της υλοποίησης.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity sync_4bit_adder_tb is
end sync_4bit_adder_tb;

architecture tb_arch of sync_4bit_adder_tb is
    component Sync_Full_Adder_4bit is
        Port ( A, B : in  std_logic_vector (3 downto 0);
               Cin : in  std_logic;
               Clock : in std_logic;
               Sum : out  std_logic_vector (3 downto 0);
               Cout : out std_logic);
    end component;

    signal a_tb : std_logic_vector(3 downto 0);
    signal b_tb : std_logic_vector(3 downto 0);
    signal sum_tb : std_logic_vector(3 downto 0);
    signal cin_tb, cout_tb, clk_tb : std_logic;

    constant CLK_PERIOD : time := 10 ns;

begin

    DUT: Sync_Full_Adder_4bit
    port map (
        A => a_tb,
        B => b_tb,
        Cin => cin_tb,
        Sum => sum_tb,
        Cout => cout_tb,
        Clock => clk_tb
    );

    clk_process: process
    begin
        while true loop
            clk_tb <= '0';
            wait for CLK_PERIOD / 2;
            clk_tb <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
    end process;

```

```
    stimulus: process
    begin
            a_tb <= "0101";
            b_tb <= "0010";
            cin_tb <= '1';
            wait for CLK_PERIOD;

            a_tb <= "0111";
            b_tb <= "1010";
            cin_tb <= '1';
            wait for CLK_PERIOD;

            a_tb <= "1011";
            b_tb <= "0011";
            cin_tb <= '0';
            wait for CLK_PERIOD;

            a_tb <= "1100";
            b_tb <= "0100";
            cin_tb <= '1';
            wait for CLK_PERIOD;

            a_tb <= "1110";
            b_tb <= "0001";
            cin_tb <= '0';
            wait for CLK_PERIOD;

            a_tb <= "1101";
            b_tb <= "1000";
            cin_tb <= '0';
            wait for CLK_PERIOD;

            a_tb <= "1001";
            b_tb <= "0110";
            cin_tb <= '1';
            wait;
    end process;
end tb_arch;
```

Το παραπάνω testbench εφαρμόζει κάποιους διαφορετικούς συνδυασμούς εισόδων στον σύγχρονο πλήρη αθροιστή 4-bit προκειμένου να εξετάσουμε τις αντίστοιχες εξόδους του. Συγκεκριμένα εκτελούνται οι παρακάτω προσθέσεις:

0101 + 0010 με Cin=1 → 1000 με Cout=0

0111 + 1010 με Cin=1 → 0010 με Cout=1

1011 + 0011 με Cin=0 → 1110 με Cout=0

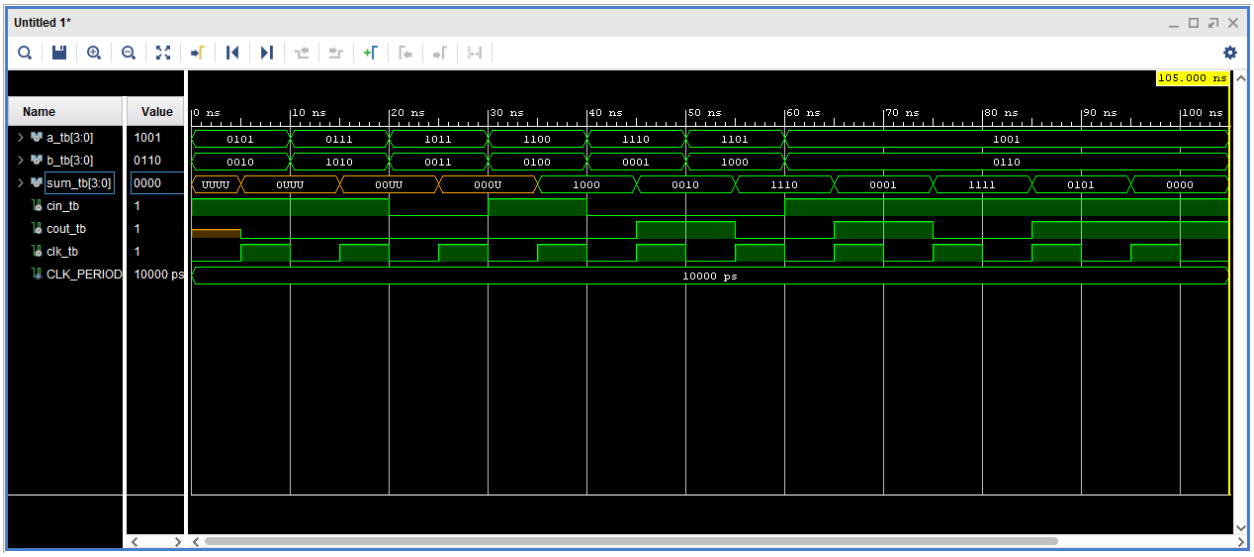
1100 + 0100 με Cin=1 → 0001 με Cout=1

1110 + 0001 με Cin=0 → 1111 με Cout=0

1101 + 1000 με Cin=0 → 0101 με Cout=1

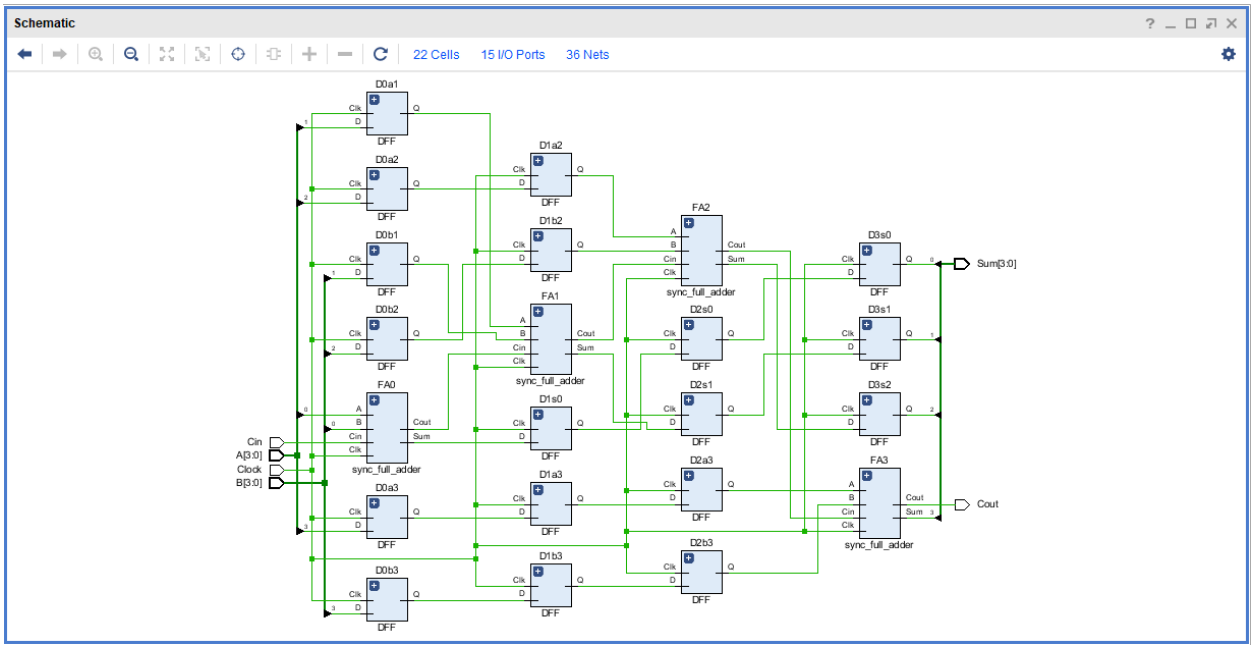
1001 + 0110 με Cin=1 → 0000 με Cout=1

Η προσομοίωση του 4-bit Synchronous Full Adder φαίνεται παρακάτω:



Παρατηρούμε ότι σε κάθε θετικό παλμό του ρολογιού παίρνουμε τις εισόδους A, B και Cin του αθροιστή και μετά από 3 κύκλους ρολογιού οι έξοδοι Sum και Cout παίρνουν κάθε φορά την αντίστοιχη τιμή που υπολογίσαμε παραπάνω. Συνεπώς, το κύκλωμα του σύγχρονου πλήρους αθροιστή 4-bit λειτουργεί ορθά και έχουμε $\mathbf{T_{latency} = 3}$ κύκλους ρολογιού.

Το RTL του 4-bit Synchronous Full Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του 4-bit Synchronous Full Adder ξεκινάει από την είσοδο FA3/Cout_reg/C και καταλήγει στην έξοδο Cout με συνολική καθυστέρηση 4.076 ns. Παρουσιάζει, όμως, ίδια συνολική καθυστέρηση με το μονοπάτι που ξεκινάει από την είσοδο D3s2/Q_reg/C και καταλήγει στην έξοδο Sum[2], και με το μονοπάτι που ξεκινάει από την είσοδο FA3/Sum_reg/C και καταλήγει στην έξοδο Sum[3].

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	2	2	1	FA3/Cout_reg/C	Cout	4.076	3.276	0.800
Path 2	∞	2	2	1	D3s2/Q_reg/C	Sum[2]	4.076	3.276	0.800
Path 3	∞	2	2	1	FA3/Sum_reg/C	Sum[3]	4.076	3.276	0.800
Path 4	∞	2	2	1	D3s0/Q_reg/C	Sum[0]	4.058	3.258	0.800
Path 5	∞	2	2	1	D3s1/Q_reg/C	Sum[1]	4.058	3.258	0.800
Path 6	∞	2	3	2	B[0]	D2s0/Q_reg_srl3/D	2.265	1.132	1.133
Path 7	∞	2	3	2	A[0]	FA0/Cout_reg/D	1.906	1.106	0.800
Path 8	∞	2	2	2	D0a1/Q_reg/C	D2s1/Q_reg_srl2/D	1.862	0.777	1.085
Path 9	∞	1	2	1	A[1]	D0a1/Q_reg/D	1.782	0.982	0.800
Path 10	∞	1	2	1	A[2]	D0a2/Q_reg/D	1.782	0.982	0.800

Η κατανάλωση πόρων του 4-bit Synchronous Full Adder είναι η εξής:

Resource	Estimation	Available	Utilization %
LUT	6	17600	0.03
LUTRAM	2	6000	0.03
FF	21	35200	0.06
IO	15	100	15.00
BUFG	1	32	3.13

Αντίστοιχα, ο Παράλληλος Αθροιστής της Εργαστηριακής Άσκησης 2 είχε κρίσιμο μονοπάτι με συνολική καθυστέρηση 5.970 ns και κατανάλωση πόρων:

Resource	Estimation	Available	Utilization %
LUT	4	17600	0.02
IO	14	100	14.00

Παρατηρούμε ότι με την τεχνική Pipeline έχουμε καταφέρει να μειώσουμε την συνολική καθυστέρησή στο κρίσιμο μονοπάτι, αφού τα μονοπάτια μεταξύ των επιπέδων είναι μικρότερα σε σχέση με τα μονοπάτια εισόδου-εξόδου. Από την άλλη, όμως, καταναλώνουμε περισσότερους πόρους, αφού χρειάζονται επιπλέον υλικά (D flip-flops και άλλα σημεία) για τον χρονισμό. Συνεπώς, η μία υλοποίηση προσφέρει ένα πιο γρήγορο και η άλλη υλοποίηση ένα πιο οικονομικό κύκλωμα.

Ζητούμενο 3: 4-bit Synchronous Multiplier

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity Sync_Mult_4bit is
5       port (A, B : in std_logic_vector(3 downto 0);
6             Clk : in std_logic;
7             Product : out std_logic_vector(7 downto 0));
8   end entity;
9
10  architecture Systolic of Sync_Mult_4bit is
11
12      component DFF is
13      port (D, Clk : in std_logic;
14            Q : out std_logic);
15      end component;
16
17      component mac is
18      port (A, B, S_in, Cin, Clk : in std_logic;
19            Sum, Cout : out std_logic);
20      end component;
21
22      signal s00 : std_logic;
23      signal c00 : std_logic;
24      signal A0, B0 : std_logic_vector(3 downto 0);
25
26      signal s01 : std_logic;
27      signal c01 : std_logic;
28      signal A1, B1 : std_logic_vector(3 downto 0);
29      signal p1 : std_logic;
30
31      signal s02, s10 : std_logic;
32      signal c02, c10 : std_logic;
33      signal A2, B2 : std_logic_vector(3 downto 0);
34      signal p2 : std_logic_vector(1 downto 0);
35
36      signal s03, s11 : std_logic;
37      signal c03, c11 : std_logic;
38      signal c03_delayed : std_logic;
39      signal A3 : std_logic_vector(3 downto 0);
40      signal B3 : std_logic_vector(3 downto 1);
41      signal p3 : std_logic_vector(1 downto 0);
42
43      signal s12, s20 : std_logic;
44      signal c12, c20 : std_logic;
45      signal A4 : std_logic_vector(3 downto 0);
46      signal B4 : std_logic_vector(3 downto 1);
47      signal p4 : std_logic_vector(2 downto 0);
48
```

```
    signal s13, s21 : std_logic;
    signal c13, c21 : std_logic;
    signal c13_delayed : std_logic;
    signal A5 : std_logic_vector(3 downto 0);
    signal B5 : std_logic_vector(3 downto 2);
    signal p5 : std_logic_vector(2 downto 0);

    signal s22, s30 : std_logic;
    signal c22, c30 : std_logic;
    signal A6 : std_logic_vector(3 downto 1);
    signal B6 : std_logic_vector(3 downto 2);
    signal p6 : std_logic_vector(3 downto 0);

    signal s23, s31 : std_logic;
    signal c23, c31 : std_logic;
    signal c23_delayed : std_logic;
    signal A7 : std_logic_vector(3 downto 2);
    signal B7 : std_logic_vector(3 downto 3);
    signal p7 : std_logic_vector(4 downto 0);

    signal s32 : std_logic;
    signal c32 : std_logic;
    signal A8, B8 : std_logic_vector(3 downto 3);
    signal p8 : std_logic_vector(5 downto 0);

begin
    --STAGE 0
    MAC00: mac port map (A(0), B(0), '0', '0', Clk, s00, c00);
    D0a0: DFF port map (A(0), Clk, A0(0));
    D0a1: DFF port map (A(1), Clk, A0(1));
    D0a2: DFF port map (A(2), Clk, A0(2));
    D0a3: DFF port map (A(3), Clk, A0(3));
    D0b0: DFF port map (B(0), Clk, B0(0));
    D0b1: DFF port map (B(1), Clk, B0(1));
    D0b2: DFF port map (B(2), Clk, B0(2));
    D0b3: DFF port map (B(3), Clk, B0(3));

    --STAGE 1
    MAC01: mac port map (A0(1), B0(0), '0', c00, Clk, s01, c01);
    D1a0: DFF port map (A0(0), Clk, A1(0));
    D1a1: DFF port map (A0(1), Clk, A1(1));
    D1a2: DFF port map (A0(2), Clk, A1(2));
    D1a3: DFF port map (A0(3), Clk, A1(3));
    D1b0: DFF port map (B0(0), Clk, B1(0));
    D1b1: DFF port map (B0(1), Clk, B1(1));
    D1b2: DFF port map (B0(2), Clk, B1(2));
    D1b3: DFF port map (B0(3), Clk, B1(3));
    D1p0: DFF port map (s00, Clk, p1);

```

```
    --STAGE 2
    MAC02: mac port map (A1(2), B1(0), '0', c01, Clk, s02, c02);
    MAC10: mac port map (A1(0), B1(1), s01, '0', Clk, s10, c10);
    D2a0: DFF port map (A1(0), Clk, A2(0));
    D2a1: DFF port map (A1(1), Clk, A2(1));
    D2a2: DFF port map (A1(2), Clk, A2(2));
    D2a3: DFF port map (A1(3), Clk, A2(3));
    D2b0: DFF port map (B1(0), Clk, B2(0));
    D2b1: DFF port map (B1(1), Clk, B2(1));
    D2b2: DFF port map (B1(2), Clk, B2(2));
    D2b3: DFF port map (B1(3), Clk, B2(3));
    D2p0: DFF port map (p1, Clk, p2(0));
    D2p1: DFF port map (s10, Clk, p2(1));

    --STAGE 3
    MAC03: mac port map (A2(3), B2(0), '0', c02, Clk, s03, c03);
    MAC11: mac port map (A2(1), B2(1), s02, c10, Clk, s11, c11);
    D3a0: DFF port map (A2(0), Clk, A3(0));
    D3a1: DFF port map (A2(1), Clk, A3(1));
    D3a2: DFF port map (A2(2), Clk, A3(2));
    D3a3: DFF port map (A2(3), Clk, A3(3));
    D3b1: DFF port map (B2(1), Clk, B3(1));
    D3b2: DFF port map (B2(2), Clk, B3(2));
    D3b3: DFF port map (B2(3), Clk, B3(3));
    D3p0: DFF port map (p2(0), Clk, p3(0));
    D3p1: DFF port map (p2(1), Clk, p3(1));
    D3c03: DFF port map (c03, Clk, c03_delayed);

    --STAGE 4
    MAC12: mac port map (A3(2), B3(1), s03, c11, Clk, s12, c12);
    MAC20: mac port map (A3(0), B3(2), s11, '0', Clk, s20, c20);
    D4a0: DFF port map (A3(0), Clk, A4(0));
    D4a1: DFF port map (A3(1), Clk, A4(1));
    D4a2: DFF port map (A3(2), Clk, A4(2));
    D4a3: DFF port map (A3(3), Clk, A4(3));
    D4b1: DFF port map (B3(1), Clk, B4(1));
    D4b2: DFF port map (B3(2), Clk, B4(2));
    D4b3: DFF port map (B3(3), Clk, B4(3));
    D4p0: DFF port map (p3(0), Clk, p4(0));
    D4p1: DFF port map (p3(1), Clk, p4(1));
    D4p2: DFF port map (s20, Clk, p4(2));

    --STAGE 5
    MAC13: mac port map (A4(3), B4(1), c03_delayed, c12, Clk, s13, c13);
    MAC21: mac port map (A4(1), B4(2), s12, c20, Clk, s21, c21);
    D5a0: DFF port map (A4(0), Clk, A5(0));
    D5a1: DFF port map (A4(1), Clk, A5(1));
    D5a2: DFF port map (A4(2), Clk, A5(2));
    D5a3: DFF port map (A4(3), Clk, A5(3));
    D5b2: DFF port map (B4(2), Clk, B5(2));
    D5b3: DFF port map (B4(3), Clk, B5(3));
    D5p0: DFF port map (p4(0), Clk, p5(0));
    D5p1: DFF port map (p4(1), Clk, p5(1));
    D5p2: DFF port map (p4(2), Clk, p5(2));
    D5c13: DFF port map (c13, Clk, c13_delayed);
```

```
    --STAGE 6
    MAC22: mac port map (A5(2), B5(2), s13, c21, Clk, s22, c22);
    MAC30: mac port map (A5(0), B5(3), s21, '0', Clk, s30, c30);
    D6a1: DFF port map (A5(1), Clk, A6(1));
    D6a2: DFF port map (A5(2), Clk, A6(2));
    D6a3: DFF port map (A5(3), Clk, A6(3));
    D6b2: DFF port map (B5(2), Clk, B6(2));
    D6b3: DFF port map (B5(3), Clk, B6(3));
    D6p0: DFF port map (p5(0), Clk, p6(0));
    D6p1: DFF port map (p5(1), Clk, p6(1));
    D6p2: DFF port map (p5(2), Clk, p6(2));
    D6p3: DFF port map (s30, Clk, p6(3));

    --STAGE 7
    MAC23: mac port map (A6(3), B6(2), c13_delayed, c22, Clk, s23, c23);
    MAC31: mac port map (A6(1), B6(3), s22, c30, Clk, s31, c31);
    D7a2: DFF port map (A6(2), Clk, A7(2));
    D7a3: DFF port map (A6(3), Clk, A7(3));
    D7b3: DFF port map (B6(3), Clk, B7(3));
    D7p0: DFF port map (p6(0), Clk, p7(0));
    D7p1: DFF port map (p6(1), Clk, p7(1));
    D7p2: DFF port map (p6(2), Clk, p7(2));
    D7p3: DFF port map (p6(3), Clk, p7(3));
    D7p4: DFF port map (s31, Clk, p7(4));
    D7c23: DFF port map (c23, Clk, c23_delayed);

    --STAGE 8
    MAC32: mac port map (A7(2), B7(3), s23, c31, Clk, s32, c32);
    D8a3: DFF port map (A7(3), Clk, A8(3));
    D8b3: DFF port map (B7(3), Clk, B8(3));
    D8p0: DFF port map (p7(0), Clk, p8(0));
    D8p1: DFF port map (p7(1), Clk, p8(1));
    D8p2: DFF port map (p7(2), Clk, p8(2));
    D8p3: DFF port map (p7(3), Clk, p8(3));
    D8p4: DFF port map (p7(4), Clk, p8(4));
    D8p5: DFF port map (s32, Clk, p8(5));

    --STAGE 9
    MAC33: mac port map (A8(3), B8(3), c23_delayed, c32, Clk, Product(6), Product(7));
    Product(0) <= p8(0);
    Product(1) <= p8(1);
    Product(2) <= p8(2);
    Product(3) <= p8(3);
    Product(4) <= p8(4);
    Product(5) <= p8(5);
end Systolic;
```

Ο κώδικας των components που χρησιμοποιήθηκαν φαίνεται παρακάτω:

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity mac is
    port (A, B, S_in, Cin, Clk : in std_logic;
          Sum, Cout : out std_logic);
end mac;

architecture Structural of mac is

    component sync_full_adder is
        port (A, B, Cin, Clk : in  std_logic;
              Sum, Cout : out  std_logic);
    end component;

    signal mult_result : std_logic;

begin
    process (A, B)
    begin
        mult_result <= A and B;
    end process;

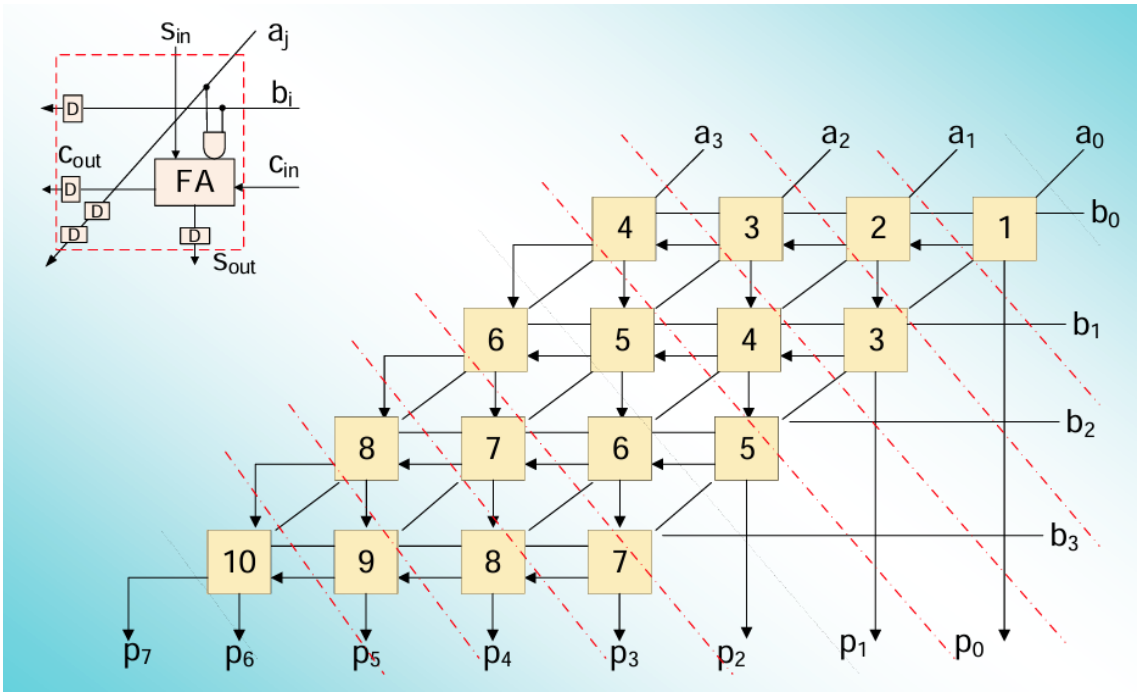
    FA: sync_full_adder port map(
        A => mult_result,
        B => S_in,
        Cin => Cin,
        Clk => Clk,
        Sum => Sum,
        Cout => Cout
    );
end Structural;
```

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity DFF is
    port (
      D   : in  std_logic;
      Clk : in  std_logic;
      Q   : out  std_logic
    );
end DFF;

architecture behavioral of DFF is
    begin
      process(Clk)
      begin
        if Clk'event and Clk='1' then
          Q <= D;
        end if;
      end process;
end behavioral;
```

Η υλοποίηση αυτού του πολλαπλασιαστή βρίσκεται στις διαφάνειες του μαθήματος και έγινε με βάση το παραπάνω σχηματικό:



Ο παραπάνω κώδικας, λοιπόν, υλοποιεί έναν σύγχρονο πολλαπλασιαστή 4-bit σε συστολική μορφή. Αρχικά, βασικό κομμάτι είναι η υλοποίηση της οντότητας του κάθε τετραγώνου που είναι το σχήμα που φαίνεται πάνω αριστερά στην εικόνα. Το component αυτό, που αποτελείται από ένα σύγχρονο πλήρη αθροιστή και μια πύλη AND, το ονομάσαμε MAC και ο κώδικας που το περιγράφει βρίσκεται πιο πάνω. Έπειτα, χωρίζουμε το κύκλωμα σε δέκα επίπεδα και βασιζόμαστε στους χρονισμούς που υπάρχουν στο παραπάνω σχήμα. Έτσι, σε κάθε επίπεδο χρησιμοποιούμε τα αντίστοιχα MAC components για να χειριστούμε τα μερικά γινόμενα του πολλαπλασιασμού και με D flip-flops περνάμε όσες πληροφορίες χρειάζονται στα επόμενα επίπεδα, είτε αυτές είναι είσοδοι είτε είναι έξοδοι των MAC. Συνολικά, εν τέλει, χρησιμοποιήσαμε 16 Synchronous Full Adders για τα 8 bit εξόδου του πολλαπλασιαστή και 83 D flip-flops για πετύχουμε τις επιθυμητές καθυστερήσεις μεταξύ όλων των επιπέδων της υλοποίησης.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity Sync_Mult_4bit_tb is
end entity;

architecture Behavioral of Sync_Mult_4bit_tb is
    component Sync_Mult_4bit is
        port (A, B : in std_logic_vector(3 downto 0);
              Clk : in std_logic;
              Product : out std_logic_vector(7 downto 0));
    end component;
    signal A_tb, B_tb : std_logic_vector(3 downto 0);
    signal P    : std_logic_vector(7 downto 0);
    signal Clk  : std_logic := '0';

begin
    DUT: Sync_Mult_4bit
        port map (
            A => A_tb,
            B => B_tb,
            Clk => Clk,
            Product => P
        );

    clk_proc: process is
    begin
        Clk <= not Clk;
        wait for 5 ns;
    end process;

    stimulus: process is
    begin

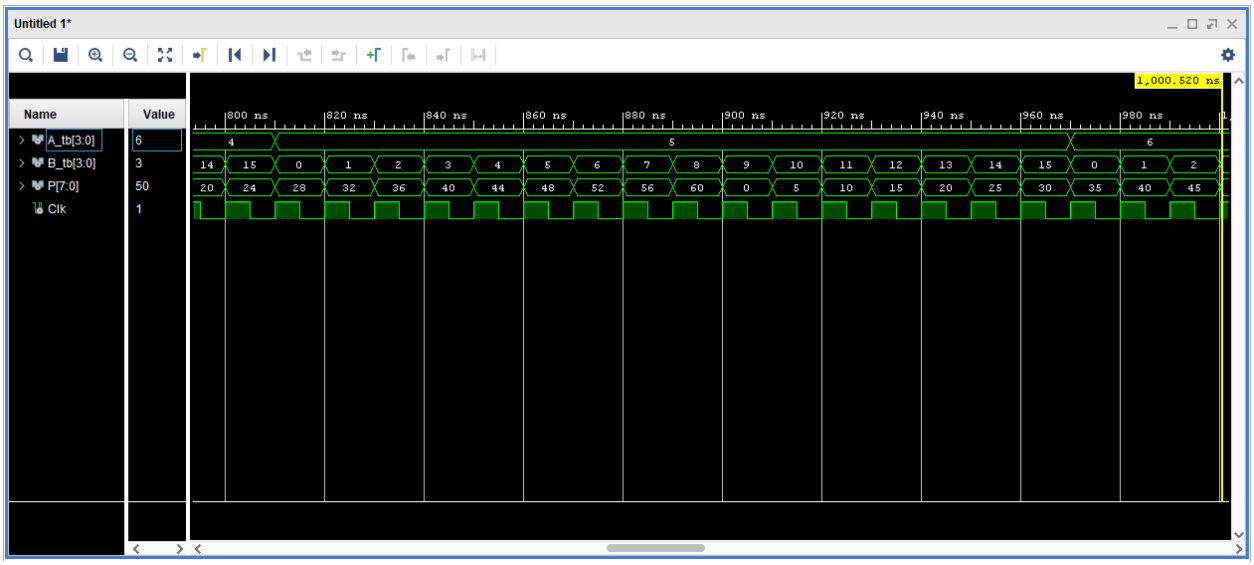
    wait for 10 ns;

    for i in 0 to 15 loop
      A_tb <= std_logic_vector(to_unsigned(i, A_tb'length));
      for j in 0 to 15 loop
        B_tb <= std_logic_vector(to_unsigned(j, B_tb'length));
        wait for 10 ns;
      end loop;
    end loop;
  end process;

end architecture;
```

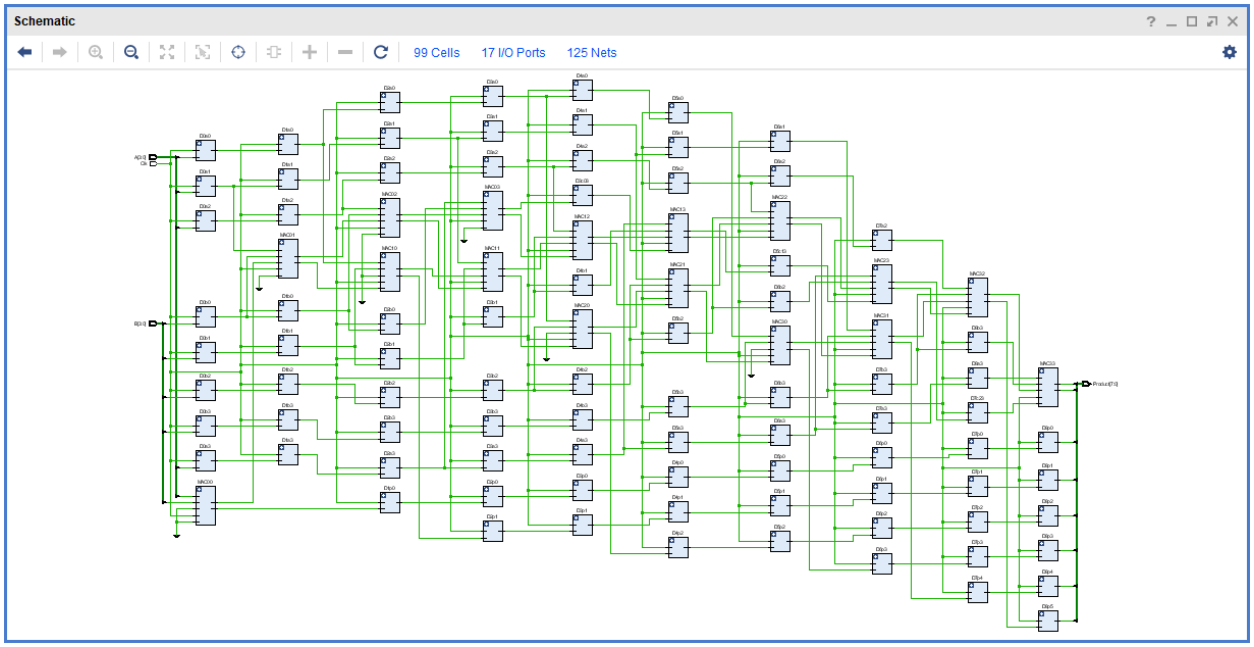
Το παραπάνω testbench εφαρμόζει στον σύγχρονο πολλαπλασιαστή 4-bit όλους τους διαφορετικούς συνδυασμούς εισόδων μέσω δύο for loops προκειμένου να εξετάσουμε τις αντίστοιχες εξόδους του.

Η προσομοίωση του 4-bit Synchronous Multiplier φαίνεται παρακάτω:



Παρατηρούμε ότι σε κάθε θετικό παλμό του ρολογιού παίρνουμε τις εισόδους A και B του πολλαπλασιαστή και μετά από 9 κύκλους ρολογιού η έξοδος P παίρνει κάθε φορά το αναμενόμενο αποτέλεσμα. Συνεπώς, το κύκλωμα του σύγχρονου πολλαπλασιαστή 4-bit λειτουργεί ορθά και έχουμε $\mathbf{T_{latency} = 9}$ κύκλους ρολογιού.

Το RTL του 4-bit Synchronous Multiplier φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του 4-bit Synchronous Multiplier ξεκινάει από την είσοδο D8p5/Q_reg/C και καταλήγει στην έξοδο Product[5] με συνολική καθυστέρηση 4.076 ns. Παρουσιάζει, όμως, ίδια συνολική καθυστέρηση με το μονοπάτι που ξεκινάει από την είσοδο MAC33/FA/Sum_reg/C και καταλήγει στην έξοδο Product[6], και με το μονοπάτι που ξεκινάει από είσοδο MAC33/FA/Cout_reg/C και καταλήγει στην έξοδο Product[7].

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	2	2	1	D8p5/Q_reg/C	Product[5]	4.076	3.276	0.800
Path 2	∞	2	2	1	MAC33/FA/Sum_reg/C	Product[6]	4.076	3.276	0.800
Path 3	∞	2	2	1	MAC33/FA/Cout_reg/C	Product[7]	4.076	3.276	0.800
Path 4	∞	2	2	1	D8p0/Q_reg/C	Product[0]	4.058	3.258	0.800
Path 5	∞	2	2	1	D8p1/Q_reg/C	Product[1]	4.058	3.258	0.800
Path 6	∞	2	2	1	D8p2/Q_reg/C	Product[2]	4.058	3.258	0.800
Path 7	∞	2	2	1	D8p3/Q_reg/C	Product[3]	4.058	3.258	0.800
Path 8	∞	2	2	1	D8p4/Q_reg/C	Product[4]	4.058	3.258	0.800
Path 9	∞	2	3	2	B[0]	D7p0/Q_reg_srl8/D	2.239	1.106	1.133
Path 10	∞	2	2	3	D6b3/Q_reg/C	D7p4/Q_reg_srl2/D	1.869	0.777	1.092